Lab 04: A Tur(n)ing Machine

ECE 484: Embedded Systems Design

Grade	Score	Signature
Attendance: Gabe Litteker		
Attendance: James Nippa		
Attendance:		
Demonstration	10	

Overview

In this lab, you will design an embedded system that will display a count that is incremented by a quadrature encoder. The outputs of the quadrature encoder will be debounced using a state machine software method and a functional system will handle 8-bit overflows in the positive and negative direction.

Deliverables

1. System Demonstration (10 points)
2. Completed Worksheet submitted to Canvas (5 points)
3. Commented Source Code submitted to Canvas (5 points)

Instructions

Your system will consist of the two seven-segment display circuits you constructed in the previous lab along with a quadrature encoder. The quadrature encoder has three terminals - A,B, and C. Channels A and B produce square waves that are 90° out of phase as shown in Figure 1. Looking at Figure 1, the channel A signal leads the channel B signal when the encoder is rotated CW. Conversely, the channel B signal leads the channel A signal when the encoder is rotated CCW. Because the two signals are created by the internal contacts of the encoder, you will need to debounce both output signals. Once the signals are debounced, you can construct a truth table by combining the previous channel states to the current channel states. The truth table is leveraged to determine the direction of the rotating dial.

TASK 1

You will connect pull-up resistors to channel A and channel B of the quadrature encoder while Pin C will be connected to ground. Using an oscilloscope, you will set up a single trigger to monitor for a falling edge on Channel A. You will then spin the dial quickly to produce a bouncing signal. Sketch the observed response at the end of the lab manual and measure the time it takes for the signal to settle and record it here:

Measured Settling Time: 100 µs

Scanned with CamScanner

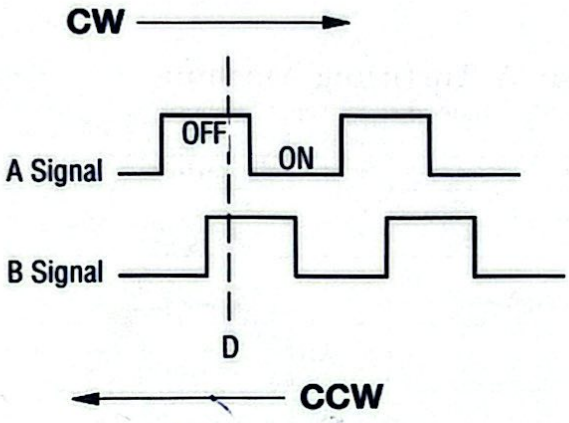


Figure 1: Quadrature Encoder Signals

TASK 2

The quadrature encoder circuitry will now be integrated with the seven-segment circuitry. Make sure to connect the middle nodes of the pull-up circuits on channel A and channel B to two available GPIO on your microcontroller. In this lab, you will be responsible for configuring Timer 1 and writing a Timer 1 ISR that will perform the following actions:

1. Read raw values of channel A and channel B every N/8 sec, where N is the measured settling time from the previous task.
2. Debounce channel signals by calling the state machine debouncing function, `softwareDebounce`. The inputs you wish to debounce should be stored in the two least significant bits of the variable you pass to `softwareDebounce`. The debounced states will be stored in the two least significant bits of the `switch_states` variable.
3. Determine which direction the encoder is rotating using the look-up table method and increment/decrement the positional count variable, `encoderCount`.
4. Handle any 8-bit overflows (8-bit overflow occurs when incrementing `encoderCount` past 0xFF and decrementing `encoderCount` past 0x00).
5. Output the `encoderCount` variable to the seven-segment displays every $\approx$ 1ms using time multiplexing on Timer 1. You will need to copy your `setSevenSegments` function from the previous lab if you are using the same wiring for the seven-segments from the previous lab.

Starter code for this lab is posted to Canvas. I have included the code for step 3 and you will be responsible for writing the remaining code within the Timer 1 ISR and the configuration code for I/O and Timer 1. Once again, **no code is permitted within the `while(1)` loop.**

0 6 0 0 0 0 0 0 0 0
7 6 5 4 3 2 1 0

Scanned with CamScanner

Demonstration Rubric

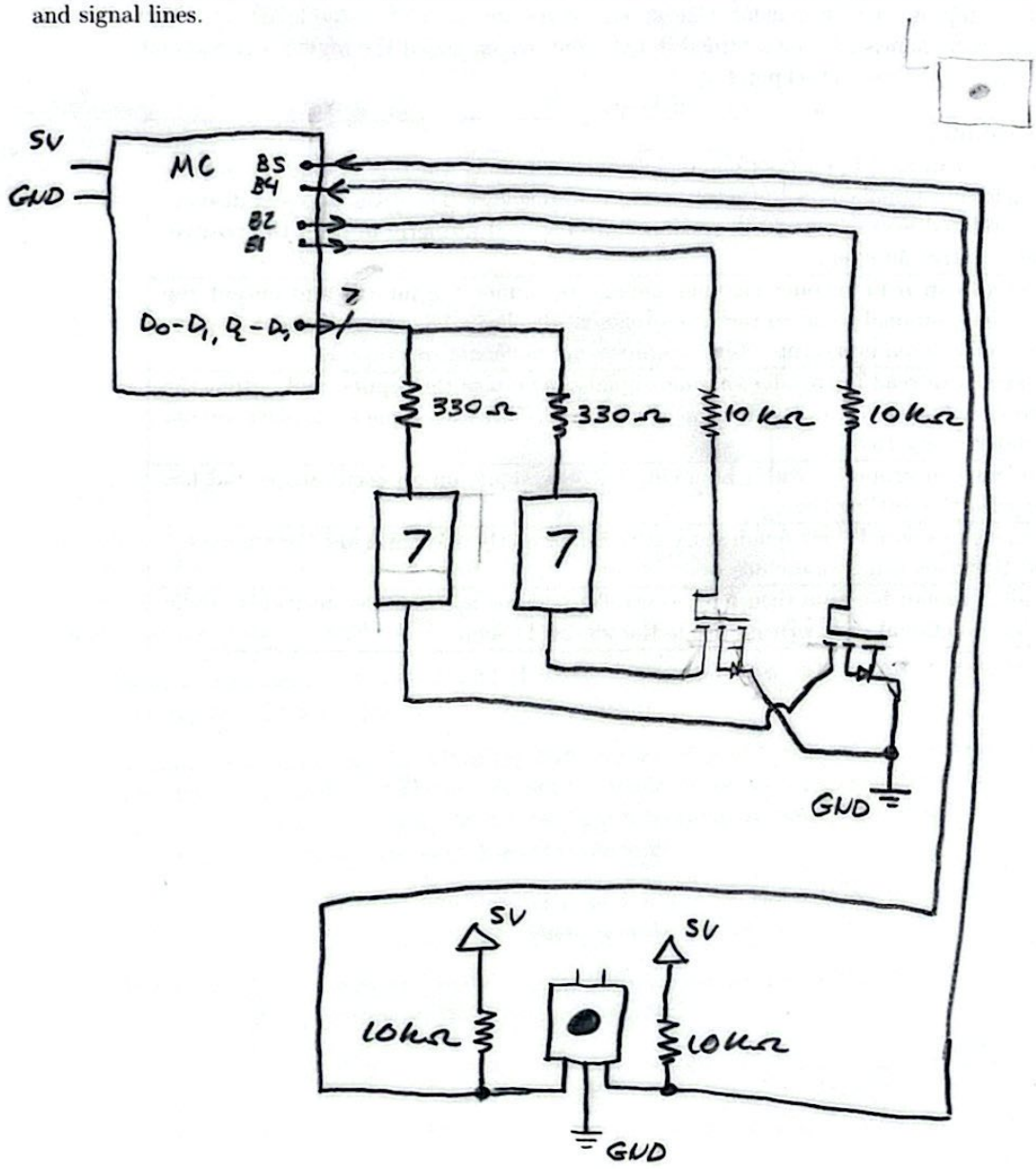
Please demonstrate your system to Dr. Swanson or the TA and they will record your demonstration score at the top of your lab manual. Checkpoint scores are provided in the following rubric. Scores between checkpoints may be awarded if a student has exceeded the previous checkpoint but has not reached the next checkpoint(s).

Score	Reasoning
10.0	Student can read in encoder channel signals, debounce the inputs, and output the encoder positional count to the seven-segment displays. The seven segment displays are updated every 1ms. 8-bit overflows are handled properly in both the positive and negative direction.
8.0	Student can read encoder channel signals, debounce the inputs, and output the encoder positional count to the seven-segment displays. The seven segment displays are not updated every 1ms. 8-bit overflows are not handled properly.
6.0	Student can read in encoder channel signals, debounce the inputs, and output the directional value to the seven-segment displays. The seven-segment displays are not updated every 1ms.
4.0	Student can properly read a bouncing encoder signal on an oscilloscope and has recorded the settling time.
2.0	Student has wired their quadrature encoder properly and can read the channel A and B signals using an oscilloscope.
0.0	Student cannot program their microcontroller and has not installed microchip studio or has functional code written inside the `while(1)` loop

Scanned with CamScanner

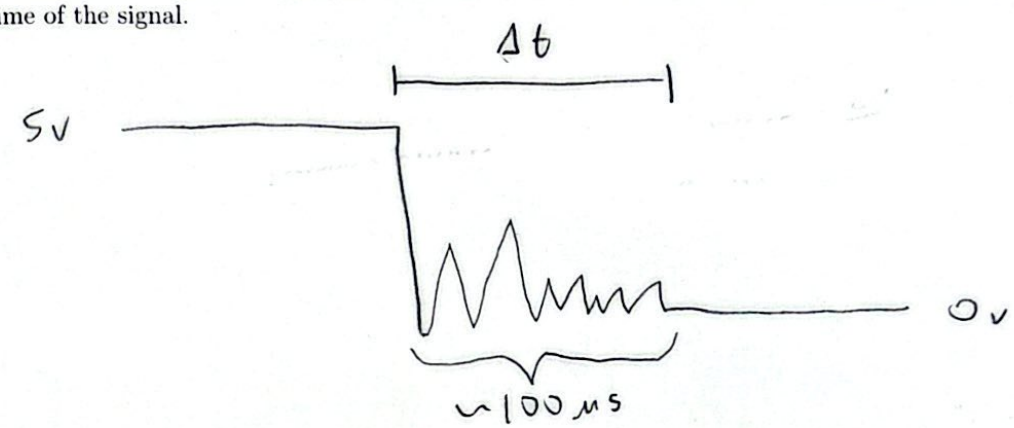
Circuit Diagram

Draw a circuit diagram for your system. Make sure to label all major modules, resistance values, and signal lines.

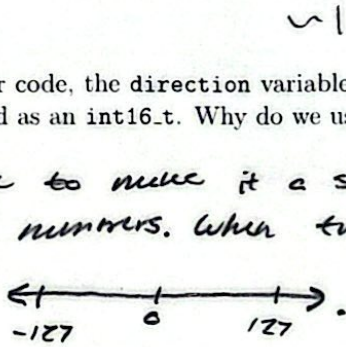


Design Questions

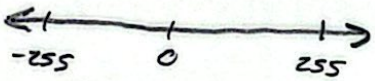
1. Record the bouncing signal you observed on the oscilloscope. Make sure to label the settling time of the signal.



2. In the starter code, the `direction` variable, which is the output from the `encoder_table` array, is declared as an `int16_t`. Why do we use `int16_t` instead of `uint8_t`?

We have to make it a signed integer as the encoder table uses negative numbers. When twos compliment is used the range for 8 bits becomes: 

. We want to represent 255. So, we need to double the bits to be 16 so the range is:



3. When you turn the dial on the encoder, there is tactile feedback in the form of a "click". This "click" equates to incrementing/decrementing the `encoderCount` variable by four. How could you adjust the software such that a single "click" equates to a single increment/decrement of `encoderCount`?

Since the encoderCount variable is a 16 bit signed integer, we can simply divide its value by 4 and display the result.

Scanned with CamScanner